

Scalability: From 1 to N Projects

Scalability: From 1 to N Projects I

The Standalone Project Paradigm enables horizontal scaling: adding a new project requires creating a directory with `manuscript/config.yaml` and nothing else. No infrastructure code changes, no `pyproject.toml` modifications, no CI configuration updates. The `run.sh` orchestrator automatically discovers new projects and presents them in its interactive menu.

We have validated scaling with 24 canonical exemplars under `projects/templates/`—always present for onboarding and tooling—and with this manuscript from `projects/templates/template_template` (147 tests) as a git-tracked public exemplar in the same automated discovery menus.

Scalability: From 1 to N Projects I

Canonical trio:

Scalability: From 1 to N Projects I

- ▶ `template_code_project`: Numerical optimization example with gradient-descent narration and 246 discovered tests. Its `pyproject.toml` declares the 90% project-source floor. Minimal footprint: compact `src/`, scripted analysis, short manuscript sections.
- ▶ `template_prose_project`: Prose-heavy manuscript emphasizing narrative structure and bibliography discipline, with 151 discovered tests and the same declared project-source floor; tests exercise rendering and Markdown integrity without heavyweight numerics.
- ▶ `template_autoresearch_project`: AutoResearch readiness workflow — bounded ML task loop (deterministic MNIST run), plan validation (`infrastucture.autoresearch`), evidence-registry reports, and manuscript hydration; its own scripts (`run_autoresearch_loop.py`, `z_generate_manuscript_variables.py`) stay thin over `src/` logic. `template_search_project` exercises literature-search workflows (remote corpus JSON fusion, `y_generate_search_figures.py`, review reports).

Scalability: From 1 to N Projects I

Meta manuscript (`projects/templates/template_template`) analyzes the repository via `src/template_template/` introspection metrics; it now lives alongside the other public exemplars under `projects/templates/`.

These workspaces share no project-level code—only Layer 1 (28 infrastructure subdirectories, ~769 Python files)—validating insulation between domain repos and reusable services.

Multi-Project Orchestration I

When the `--all-projects` flag is passed to `run.sh`, the pipeline executes each discovered project sequentially, running infrastructure tests once at the start and skipping them for individual projects to avoid redundant validation. After all projects complete, a cross-project executive report aggregates metrics (test counts, coverage percentages, page counts, rendering durations) into a unified dashboard with both JSON and Markdown output formats. This executive reporting stage provides repository-level visibility without requiring any project-specific reporting code.

Scaling Metrics I

Scaling Metrics I

Metric	template_code_project	template_prose_project	template_autoresearch_project
Source modules	26	8	60
Test files	12	12	21
Test count	246	151	307
Manuscript chapters	9	8	6
Analysis scripts	10	4	5
Figures (auto-generated)	9	5	27

The infrastructure overhead per project is constant regardless of project size: the same 28 modules, the same 14 pipeline stages, the same rendering and validation logic. This $O(1)$ infrastructure cost is the architectural payoff of the Two-Layer separation.